Getting Started with LatinCy

Patrick J. Burns

2024-08-26

Table of contents

Preface **5**
Key links . . . 5
Acknowledgements . . . 5

Abbreviations **6**

I Front Matter **7**

II Model/Pipeline Basics **8**

1 Installing LatinCy models **9**
1.1 Installing spaCy . . . 9
1.2 Installing the LatinCy models . . . 9
1.3 Installing additional packages . . . 9

2 Loading LatinCy models **11**
2.1 Loading LatinCy models with spaCy . . . 11
2.2 Creating a spaCy doc from a string . . . 11

3 LatinCy components and their annotations **13**
3.1 LatinCy components and their annotations . . . 13

III NLP Tasks **15**

4 Sentence segmentation **16**
4.1 Sentence segmentation with LatinCy . . . 16
References . . . 19

5 Word Tokenization **20**
5.1 Word tokenization with LatinCy . . . 20
5.1.1 Token attributes and methods related to tokenization . . . 22
5.1.2 Customization of the spaCy tokenizer in LatinCy . . . 25
5.1.3 Normalization at the tokenization stage . . . 28

References . . . 28

6 Lemmatization **29**
6.1 Lemmatization with LatinCy . . . 29
References . . . 36

7 POS Tagging **37**
7.1 POS Tagging with LatinCy . . . 37
References . . . 42

8 Morphological Tagging **43**
8.1 Morphological Tagging with LatinCy . . . 43
8.1.1 The `remorpher` component . . . 45
References . . . 46

9 Named Entity Recognition **47**
9.1 Named Entity Recognition with LatinCy . . . 47
References . . . 50

10 Vectors **51**
10.1 Accessing Vectors with LatinCy . . . 51
10.1.1 Static vectors are context-independent . . . 53
10.1.2 Contextual vectors from tok2vec . . . 54
10.1.3 Span and document vectors . . . 54
References . . . 55

11 TRF Vectors **56**
11.1 Accessing TRF Vectors with LatinCy . . . 56
References . . . 59

12 Noun Chunking **60**
12.1 Noun Chunking with LatinCy . . . 60
References . . . 61

13 Syntax iterators **62**
13.1 Other kinds of "chunking" with LatinCy . . . 62
13.1.1 Extracting *cum* clauses with LatinCy . . . 62
13.1.2 Extracting prepositional phrases with LatinCy . . . 67

IV spaCy Tasks **70**

14 Sequence Matching **71**
14.1 Sequence matching with LatinCy 71
References 77

15 Visualizing Annotations **78**
15.1 Visualizing Annotations with DisplaCy 78
15.1.1 Visualizing dependency parses 78
15.1.2 Visualizing named entities 79
References 81

Appendices **82**

A Open LatinCy Projects **82**
A.1 Model-based enclitic splitting 82

B Sample Texts **83**
B.1 Livy, Ab Urbe Condita "Praefatio" 83

References **85**

Preface

"Getting Started with LatinCy" is an always-a-work-in-progress combination of documentation and demo notebooks for working with the LatinCy models on a variety of Latin text analysis and NLP tasks.

Key links

Models: https://huggingface.co/latincy
Universe: https://spacy.io/universe/project/latincy
Preprint: https://arxiv.org/abs/2305.04365

This book has been written using Jupyter notebooks which have then been collated with Quarto. To learn more about Quarto books visit https://quarto.org/docs/books.

Acknowledgements

Thank you to all of the readers who have submitted corrections via GitHub PRs: @sjhuskey.

Abbreviations

Where possible, I will include references to standard NLP works using the following abbreviations:

MS Mesquita, D., and Altinok, D. 2025. Mastering spaCy. 2nd ed. Birmingham, U.K.: Packt Publishing Ltd. repo (Mesquita and Altinok 2025)
NLTK Bird, S., Klein, E., and Loper, E. 2015. *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. 2nd Edition. https://www.nltk.org/book/. (Bird, Klein, and Loper 2015)
SLP Jurafsky, D., and Martin, J.H. 2026. *Speech and Language Processing*. 3rd Edition, Draft. https://web.stanford.edu/~jurafsky/slp3/. (Jurafsky and Martin 2026)

Part I

Front Matter

Part II

Model/Pipeline Basics

1 Installing LatinCy models

1.1 Installing spaCy

The LatinCy models are designed to work with the spaCy natural language platform, so you will need to have this package installed before anything else. The following cell has the pip install command for spaCy. At the time of writing, the latest version available for spaCy compatible with LatinCy is v3.8.4.

NB: To run the cells below, uncomment the commands by removing the # at the beginning of the line. The exclamation point at the beginning of the line is shorthand for the `%system` magic command in Jupyter which can be used to run shell commands from within a notebook.

```
# !pip install -U spacy
```

1.2 Installing the LatinCy models

LatinCy models are currently available in three sizes: 'sm', 'md', and 'lg'. We will use the different models throughout the tutorials, so let's install all three now so that they are available for future chapters.

```
# !pip install "la-core-web-sm @ https://huggingface.co/latincy/la_core_web_sm/resolve/main/l
# !pip install "la-core-web-md @ https://huggingface.co/latincy/la_core_web_md/resolve/main/l
# !pip install "la-core-web-lg @ https://huggingface.co/latincy/la_core_web_lg/resolve/main/l
# !pip install "la-core-web-trf @ https://huggingface.co/latincy/la_core_web_trf/resolve/mai
```

1.3 Installing additional packages

We will also use other packages throughout these tutorials. They are included here for your convenience, as well as in the `requirements.txt` file in the code repository for this Quarto book.

```
# !pip install pandas
# !pip install matplotlib
# !pip install seaborn
# !pip install scikit-learn
# !pip install tqdm
```

2 Loading LatinCy models

2.1 Loading LatinCy models with spaCy

```
# Imports

import spacy
from pprint import pprint
```

```
nlp = spacy.load('la_core_web_lg')
```

2.2 Creating a spaCy doc from a string

```
text = "Haec narrantur a poetis de Perseo."
doc = nlp(text)
print(doc)
```

```
Haec narrantur a poetis de Perseo .
```

```
print(type(doc))
```

```
<class 'spacy.tokens.doc.Doc'>
```

```
print(doc.__repr__())
```

```
Haec narrantur a poetis de Perseo .
```

```
print(doc.text)
```

```
Haec narrantur a poetis de Perseo .
```

```python
print(type(doc.text))
```

```
<class 'str'>
```

3 LatinCy components and their annotations

3.1 LatinCy components and their annotations

```
# Imports & setup

import spacy
import pandas as pd
from pprint import pprint
nlp = spacy.load('la_core_web_lg')
text = "avus eius Acrisius appellabatur."
doc = nlp(text)
print(doc)
```

```
auus eius Acrisius appellabatur .
```

Here are the components provided by the default LatinCy models…

```
pprint(nlp.pipe_names)
```

```
['tok2vec',
 'senter',
 'normer',
 'tagger',
 'morphologizer',
 'trainable_lemmatizer',
 'lookup_lemmatizer',
 'uv_normalizer',
 'parser',
 'harmonizer',
 'remorpher',
 'ner']
```

The dataframe below summarizes the key annotations provided by these components…

```
data = []

for token in doc:
    data.append([token.text, token.norm_, token.lemma_, token.pos_, token.tag_, token.morph.

df = pd.DataFrame(data, columns=['text', 'norm', 'lemma', 'pos', 'tag', 'morph', 'dep', 'ent

df
```

	text	norm	lemma	pos	tag	morph
0	auus	auus	auus	NOUN	noun	Case=Nom|Gender=Masc|Number=Sin
1	eius	eius	is	PRON	pronoun	Case=Gen|Gender=Fem|Number=Sing
2	Acrisius	acrisius	Acrisius	PROPN	proper_noun	Case=Nom|Gender=Masc|Number=Sin
3	appellabatur	appellabatur	appello	VERB	verb	Aspect=Imp|Mood=Ind|Number=Sing|
4	.	.	.	PUNCT	punc	

Part III

NLP Tasks

4 Sentence segmentation

4.1 Sentence segmentation with LatinCy

Sentence segmentation is the task of splitting a text into sentences. For the LatinCy models, this is a task been trained using spaCy's `senter` factory to terminate sentences at both strong and weak stops, following the example of Clayman (1981) (see also, Wake (1957), Janson (1964)), who writes: "If all stops are made equivalent, i.e. if no distinction is made between the strong stop, weak stop and interrogation mark, editorial differences will be kept to a minimum."

Given a spaCy `Doc`, the `sents` attribute will produce a `generator` object with the sentence from that document as determined by the dependency parser. Each sentence is a `Span` object with the start and end token indices from the original `Doc`.

```
# Imports & setup

import spacy
from pprint import pprint
nlp = spacy.load('la_core_web_lg')
text = "Haec narrantur a poetis de Perseo. Perseus filius erat Iovis, maximi deorum. Avus eiu
doc = nlp(text)
print(doc)
```

```
Haec narrantur a poetis de Perseo . Perseus filius erat Iouis , maximi deorum . Auus eius Ac
```

```
sents = doc.sents
print(type(sents).__name__)
```

```
generator
```

Like all `Span` objects, the text from each sentence can be retrieved with the `text` attribute. For convenience below, we convert the generator to list so that we can iterate over it multiple times. Here are the three (3) sentences identified in the example text as well as an indication of the sentences' `type`, i.e. `<class 'spacy.tokens.span.Span'>`.

```
sents = list(sents)

for i, sent in enumerate(sents, 1):
    print(f'{i}: {sent.text}')
```

```
1: Haec narrantur a poetis de Perseo .
2: Perseus filius erat Iouis , maximi deorum .
3: Auus eius Acrisius appellabatur .
```

```
sent = sents[0]
print(type(sent))
```

```
<class 'spacy.tokens.span.Span'>
```

Sentences have the same atrributes/methods available to them as any span (listed in the next cell). Following are some attibutes/methods that may be particularly relevant to working with sentences.

```
sent_methods = [item for item in dir(sent) if '_' not in item]
pprint(sent_methods)
```

```
['conjuncts',
 'doc',
 'end',
 'ents',
 'id',
 'label',
 'lefts',
 'rights',
 'root',
 'sent',
 'sentiment',
 'sents',
 'similarity',
 'start',
 'subtree',
 'tensor',
 'text',
 'vector',
 'vocab']
```

You can identify the `root` of the sentence as determined by the dependency parser. Assuming the parsing in correct, this will be the main verb of the sentence.

```
print(sent.root)
```

```
narrantur
```

Each word in the sentence has an associated vector. Sentence (any `Span` in fact) has an associated vector as well that is the mean of the vectors of the words in the sentence. As this example uses the `lg` model, the vector has a length of 300.

```
print(sent.vector.shape)
```

```
(300,)
```

This vector then can be used to compute the similarity between two sentences. Here we see our example sentence compared to two related sentence: 1. a sentence where the character referred to is changed from Perseus to Ulysses; and 2. the active-verb version of the sentence.

```
sent.similarity(nlp('Haec narrantur a poetis de Ulixe.'))
```

```
0.9865403175354004
```

```
sent.similarity(nlp('Haec narrant poetae de Perseo.'))
```

```
0.8239373564720154
```

We can retrieve the start and end indices from the original document for each sentence.

```
sent_2 = sents[1]
start = sent_2.start
end = sent_2.end
print(start)
print(end)
print(sent_2.text)
print(doc[start:end].text)
```

```
7
15
Perseus filius erat Iouis , maximi deorum .
Perseus filius erat Iouis , maximi deorum .
```

References

SLP Chapter 2, Section 2.8.1 "Sentence Segmentation" link
spaCy SentenceRecognizer

5 Word Tokenization

5.1 Word tokenization with LatinCy

```
# Imports & setup

import spacy
from pprint import pprint
nlp = spacy.load('la_core_web_sm')
text = "Haec narrantur a poetis de Perseo. Perseus filius erat Iovis, maximi deorum. Avus eiu
doc = nlp(text)
print(doc)
```

```
Haec narrantur a poetis de Perseo . Perseus filius erat Iouis , maximi deorum . Auus eius Ac
```

Word tokenization is the task of splitting a text into words (and wordlike units like punctuation, numbers, etc.). For the LatinCy models, tokenization is the fundamental pipeline component on which all other components depend. SpaCy uses non-destructive, "canonical" tokenization, i.e. non-destructive, in that the original text can be untokenized, so to speak, based on `Token` annotations and canonical in that indices are assigned to each token during this process and these indices are used to refer to the tokens in other annotations. (Tokens *can* be separated or merged, but this requires the user to actively undo and redefine the tokenization output.) LatinCy uses a modified version of the default spaCy tokenizer that recognizes and splits enlitic *-que* using a rules-based process. (NB: It is in the LatinCy development plan to move enclitic splitting to a separate post-tokenization component.)

The spaCy `Doc` object is an iterable and tokens are the iteration unit.

```
tokens = [item for item in doc]
print(tokens)
```

```
[Haec, narrantur, a, poetis, de, Perseo, ., Perseus, filius, erat, Iouis, ,, maximi, deorum,
```

```python
token = tokens[0]
print(type(token))
```

```
<class 'spacy.tokens.token.Token'>
```

The text content of a `Token` object can be retrieved with the `text` attribute.

```python
for i, token in enumerate(tokens, 1):
    print(f'{i}: {token.text}')
```

```
1: Haec
2: narrantur
3: a
4: poetis
5: de
6: Perseo
7: .
8: Perseus
9: filius
10: erat
11: Iouis
12: ,
13: maximi
14: deorum
15: .
16: Auus
17: eius
18: Acrisius
19: appellabatur
20: .
```

Note again that the token itself is a spaCy `Token` object and that the `text` attribute returns a Python string even though their representations in the Jupyter Notebook look the same.

```python
token = tokens[0]
print(f'{type(token)} -> {token}')
print(f'{type(token.text)} -> {token.text}')
```

```
<class 'spacy.tokens.token.Token'> -> Haec
<class 'str'> -> Haec
```

5.1.1 Token attributes and methods related to tokenization

Here are some atrributes/methods available for spaCy `Token` objects that are relevant to word tokenization.

SpaCy keeps track of both the token indices and the character offsets within a doc using either the `i` or `idx` attributes, respectively…

```
print(token.doc)
```

```
Haec narrantur a poetis de Perseo . Perseus filius erat Iouis , maximi deorum . Auus eius Ac
```

```
# token indices

for token in doc:
    print(f'{token.i}: {token.text}')
```

```
0: Haec
1: narrantur
2: a
3: poetis
4: de
5: Perseo
6: .
7: Perseus
8: filius
9: erat
10: Iouis
11: ,
12: maximi
13: deorum
14: .
15: Auus
16: eius
17: Acrisius
18: appellabatur
19: .
```

This is functionally equivalent to using `enumerate`…

```python
# token indices, with enumerate

for i, token in enumerate(doc):
    print(f'{i}: {token.text}')
```

```
0: Haec
1: narrantur
2: a
3: poetis
4: de
5: Perseo
6: .
7: Perseus
8: filius
9: erat
10: Iouis
11: ,
12: maximi
13: deorum
14: .
15: Auus
16: eius
17: Acrisius
18: appellabatur
19: .
```

Another indexing option is the `idx` attribute which is the character offset of the token in the original `Doc` object.

```python
# character offsets,
for token in doc:
    print(f'{token.idx}: {token.text}')
```

```
0: Haec
5: narrantur
15: a
17: poetis
24: de
27: Perseo
34: .
36: Perseus
```

```
44: filius
51: erat
56: Iouis
62: ,
64: maximi
71: deorum
78: .
80: Auus
85: eius
90: Acrisius
99: appellabatur
112: .
```

Observe these `idx` attributes relate to the character offsets from the original `Doc`. To illustrate the point, we will replace spaces with an underscore in the output. We can see from the output above that *narrantur* begins at `idx` 5 and that the next word *a* begins at `idx` 15. Yet *narrantur* is only 9 characters long and the difference between these two numbers is 10! This is because we need to account for whitespace in the original `Doc`. This is handled by the attribute `text_with_ws`.

```
print(doc.text[5:15].replace(' ', '_'))
```

```
narrantur_
```

```
print(f'text -> {doc[1].text} (length {len(doc[1].text)})')
print()
print(f'text_with_ws -> {doc[1].text_with_ws} (length {len(doc[1].text_with_ws)})')
```

```
text -> narrantur (length 9)

text_with_ws -> narrantur  (length 10)
```

Accordingly, using the `text_with_ws` attribute (as opposed to simply the `text` attribute) we can reconstruct the original text. This is what was meant above by "non-destructive" tokenization. Look at the difference between a text joined using the `text` attribute and one joined using the `text_with_ws` attribute.

```
joined_tokens = ' '.join([token.text for token in doc])
print(joined_tokens)
print(joined_tokens == doc.text)

print()

reconstructed_text = ''.join([token.text_with_ws for token in doc])
print(reconstructed_text)
print(reconstructed_text == doc.text)
```

```
Haec narrantur a poetis de Perseo . Perseus filius erat Iouis , maximi deorum . Auus eius Ac
True

Haec narrantur a poetis de Perseo . Perseus filius erat Iouis , maximi deorum . Auus eius Ac
True
```

Because spaCy tokenization is set from the outset, you can traverse the tokens in a `Doc` objects from the tokens themselves using the `nbor` method. This method takes an integer argument that specifies the number of tokens to traverse. A positive integer traverses the tokens to the right, a negative integer traverses the tokens to the left.

```
print(doc[:6])
print('-----')
print(f'{doc[3]}, i.e. i = 3')
print(f'{doc[3].nbor(-1)}, i.e. i - 1 = 2')
print(f'{doc[3].nbor(-2)}, i.e. i - 2 = 1')
print(f'{doc[3].nbor(1)}, i.e. i + 1 = 4')
print(f'{doc[3].nbor(2)}, i.e. i + 2 = 5')
```

```
Haec narrantur a poetis de Perseo
-----
poetis, i.e. i = 3
a, i.e. i - 1 = 2
narrantur, i.e. i - 2 = 1
de, i.e. i + 1 = 4
Perseo, i.e. i + 2 = 5
```

5.1.2 Customization of the spaCy tokenizer in LatinCy

LatinCy aims to tokenize the *que* enclitic in Latin texts. As noted above this is currently done through a rule-based approach. Here is the custom tokenizer code (beginning at this

line in the code) followed by a description of the process. Note that this process is based on the following recommendations in the spaCy documentation: https://spacy.io/usage/training#custom-tokenizer.

```
from spacy.util import registry, compile_suffix_regex

@registry.callbacks("customize_tokenizer")
def make_customize_tokenizer():
    def customize_tokenizer(nlp):
        suffixes = nlp.Defaults.suffixes + [
            "que",
            "qve",
        ]
        suffix_regex = compile_suffix_regex(suffixes)
        nlp.tokenizer.suffix_search = suffix_regex.search

        for item in que_exceptions:
            nlp.tokenizer.add_special_case(item, [{"ORTH": item}])
            nlp.tokenizer.add_special_case(item.lower(), [{"ORTH": item.lower()}])
            nlp.tokenizer.add_special_case(item.title(), [{"ORTH": item.title()}])
            nlp.tokenizer.add_special_case(item.upper(), [{"ORTH": item.upper()}])

    return customize_tokenizer
```

Basically, we treat *que* (and its case and u/v norm variants) as punctuation. These are added to the `Defaults.suffixes`. If no other intervention were made, then any word ending in *que* or a variant would be split into a before-*que* part and *que*. Since there are large number of relatively predictable words that end in *que*, these are maintained in a list called `que_exceptions`. All of the words in the `que_exceptions` list are added as a "special case" using the tokenizer's `add_special_case` method and so will not be split. The `que_exceptions` lists is as follows:

```
que_exceptions = ['quisque', 'quidque', 'quicque', 'quodque', 'cuiusque', 'cuique', 'quemque
```

You can see these words in the `rules` attribute of the tokenizer.

```
# Sample of 10 que rules from the custom tokenizer

tokenizer_rules = nlp.tokenizer.rules
print(sorted(list(set([rule.lower() for rule in tokenizer_rules if 'que' in rule])))[:10])
```

```
['absque', 'abusque', 'adaeque', 'adusque', 'aeque', 'antique', 'atque', 'circumundique', 'co
```

With the exception of basic enclitic splitting, the LatinCy tokenizer is the same as the default spaCy tokenizer. The default spaCy tokenizer is described in detail in the spaCy documentation. Here are some useful attributes/methods for working with LatinCy.

Tokenize a string without any other pipeline annotations with a call.

```
tokens = nlp.tokenizer(text)
print(tokens)
print(tokens[0].text)
print(tokens[0].lemma_) # Note that there is no annotation here because since the tokenizer
```

```
Haec narrantur a poetis de Perseo . Perseus filius erat Iouis , maximi deorum . Auus eius Ac
Haec
```

A list of texts can be tokenized in one pass with the `pipe` method. This yields a generator object where each item is `Doc` object of tokenized-only texts

```
texts = ["Haec narrantur a poetis de Perseo. Perseus filius erat Iovis, maximi deorum; avus

tokens = list(nlp.tokenizer.pipe(texts))

print(len(tokens)) # number of documents
print(len(tokens[0])) # number of tokens in first document
```

```
3
76
```

You can get an explanation of the tokenization "decisions" using the `explain` method. In the example below, we see how the *que* in *virumque* is treated as a suffix (as discussed above) and so is split during tokenization.

```
tok_exp = nlp.tokenizer.explain('arma virumque cano')
print(tok_exp)
```

```
[('TOKEN', 'arma'), ('TOKEN', 'virum'), ('SUFFIX', 'que'), ('TOKEN', 'cano')]
```

```
tokens = nlp.tokenizer('arma uirumque cano')
for i, token in enumerate(tokens):
    print(f'{i}: {token.text}')
```

```
0: arma
1: uirum
2: que
3: cano
```

5.1.3 Normalization at the tokenization stage

In order to start the pipeline processing with text data that is as close as possible to the data used for training all of the downstream components, the LatinCy models now include a certain amount of normalization at the tokenization stage. These four normalization functions run at tokenization: 1. macron removal; 2. accent removal; 3. ligature separation; and 4. space adjustment (i.e. converting all whitespace to a single space and stripping whitespace from the left and right side of the input text).

```
text = " Hæc     peritè\n\nnarrāntur\t\ta poētīs dē Perseō Acrisiōque."
doc = nlp(text)

print("Before normalization:")
print(text, "\n")

print("After normalization:")
print(doc.text)
```

```
Before normalization:
 Hæc     peritè

narrāntur       a poētīs dē Perseō Acrisiōque.

After normalization:
Haec perite narrantur a poetis de Perseo Acrisioque .
```

References

SLP Chapter 2, Section 2.8 "Rule-based tokenization" link

6 Lemmatization

6.1 Lemmatization with LatinCy

Lemmatization is the task of mapping a token in a text to its dictionary headword. With the default LatinCy pipelines, two components are used to perform this task: 1. spaCy's `Edit Tree Lemmatizer` and 2. a second custom `Lookup Lemmatizer`, named in the pipeline "trainable_lemmatizer" and "lookup_lemmatizer" respectively. In the first lemmatization pass, a probabilistic tree model is used to predict the transformation from the token form to its lemma. A second pass is made at the end of the pipeline which checks the token form against a large (~1M item) lemma dictionary (i.e. lookups) for ostensibly unambiguous forms; if a match is found, the lemma is overwritten with the corresponding value from lookup. The two-pass logic largely follows the approach recommended in Burns (2018) and Burns (2020), as facilitated by the spaCy pipeline architecture.

```
# Imports & setup

import spacy
from pprint import pprint
nlp = spacy.load('la_core_web_sm')
text = "Haec narrantur a poetis de Perseo. Perseus filius erat Iovis, maximi deorum. Avus ei
doc = nlp(text)
print(doc)
```

```
Haec narrantur a poetis de Perseo . Perseus filius erat Iouis , maximi deorum . Auus eius Ac
```

Note here the two lemmatizer components that are included in the pipeline, i.e. "trainable_lemmatizer" and "lookup_lemmatizer"…

```
print(nlp.pipe_names)
```

```
['tok2vec', 'senter', 'normer', 'tagger', 'morphologizer', 'trainable_lemmatizer', 'lookup_le
```

Once a text is annotated using the LatinCy pipeline, i.e. as part of the `Doc` creation process, lemmas can be found as annotations of the `Token` objects…

```
sample_token = doc[0]

print(f'Sample token: {sample_token.text}')
print(f'Sample lemma: {sample_token.lemma_}')
```

```
Sample token: Haec
Sample lemma: hic
```

```
import tabulate

data = []

tokens = [item for item in doc]

for token in tokens:
    data.append([token.text, token.lemma_])

print(tabulate.tabulate(data, headers=['Text', 'Lemma']))
```

```
Text          Lemma
------------  --------
Haec          hic
narrantur     narro
a             ab
poetis        poeta
de            de
Perseo        Perseus
.             .
Perseus       Perseus
filius        filius
erat          sum
Iouis         Iuppiter
,             ,
maximi        magnus
deorum        deus
.             .
Auus          Auus
eius          is
```

```
Acrisius      Acrisius
appellabatur  appello
.             .
```

The `lemma_` attribute has the type `str` and so is compatible with all string operations…

```
print(f'Token: {tokens[0].text}')
print(f'Lemma: {tokens[0].lemma_}')
print(f'Lowercase lemma: {tokens[0].lemma_.lower()}')
```

```
Token: Haec
Lemma: hic
Lowercase lemma: hic
```

The `lemma_` attribute, though, is only the human-readable version of the lemma. Internally, spaCy uses a hash value to represent the lemma, which is stored in the `lemma` attribute. (Note the lack of trailing underscore.)

```
print(f'Token: {tokens[1].text}')
print(f'Human-readable lemma: {tokens[1].lemma_}')
print(f'spaCy lemma key: {tokens[1].lemma}')
```

```
Token: narrantur
Human-readable lemma: narro
spaCy lemma key: 11361982710182407617
```

In order to compare the two different lemmatization passes, we can create two copies of the LatinCy pipeline, each with one of the two lemmatizers removed…

```
import copy

P1 = copy.deepcopy(nlp)
P1.disable_pipes((['tagger', 'morphologizer', 'lookup_lemmatizer']))
print(f'First pipeline components: {P1.pipe_names}')

P2 = copy.deepcopy(nlp)
P2.disable_pipes((['tagger', 'morphologizer', 'trainable_lemmatizer']))
print(f'Second pipeline components: {P2.pipe_names}')
```

```
First pipeline components: ['tok2vec', 'senter', 'normer', 'trainable_lemmatizer', 'uv_normal
Second pipeline components: ['tok2vec', 'senter', 'normer', 'lookup_lemmatizer', 'uv_normaliz
```

We can then run the same text through both pipelines and compare the results side-by-side…

```
P1_annotations = P1(text)
P2_annotations = P2(text)

data = []

for p1_token, p2_token in zip(P1_annotations, P2_annotations):
    data.append([p1_token.text, p1_token.lemma_, p2_token.lemma_])

print(tabulate.tabulate(data, headers=['Text', 'Trainable lemmatizer', 'Lookup lemmatizer']))
```

```
Text          Trainable lemmatizer    Lookup lemmatizer
------------  ----------------------  -------------------
Haec          hic                     hic
narrantur     narro                   narro
a             ab
poetis        poeta                   poeta
de            de                      de
Perseo        Perseus
.             .                       .
Perseus       Perseus                 perseus
filius        filius                  filius
erat          sum                     sum
Iouis         Iuppiter
,             ,                       ,
maximi        magnus
deorum        deus                    deus
.             .                       .
Auus          Auus                    auus
eius          is                      is
Acrisius      Acrisius
appellabatur  appello                 appello
.             .                       .
```

Even if the lookup lemmatizer overwrites the lemma predicted by the trainable lemmatizer, the original lemma predicted by the trainable lemmatizer is still stored in a custom Token attribute, i.e.

```
print(f'Token: {tokens[10].text}')
print(f'Lemma: {tokens[10].lemma_}')
print(f'Predicted lemma: {tokens[10]._.predicted_lemma}')
```

```
Token: Iouis
Lemma: Iuppiter
Predicted lemma: Iuppiter
```

Note specifically the lemmatization of *Iovis*—since it is a highly irregular form, it is not surprising that the Edit Tree Lemmatizer has manufactured a potential, but incorrect, lemma based on the root *Iov-*. Since *Iovis* is an unambiguous Latin form and has been added to the LatinCy lookups, the Lookup Lemmatizer steps in to correct the erroneous first pass. The lookup data can be found in the `la-latincy-lookups` package [https://github.com/latincy/la-latincy-lookups]. These lookups are installed as a dependency of each of the LatinCy pipelines.

The code below shows how to access the lookup data directly…

```
# Load the lookups data

import json
from la_latincy_lookups import la as la_lookups

with open(la_lookups["lemma_lookup"], "r", encoding="utf-8") as f:
    LOOKUPS = json.load(f)

print(f"Total lookup entries: {len(LOOKUPS)}")
print(f"LOOKUPS['Ioue'] = {LOOKUPS['Ioue']}")
```

```
Total lookup entries: 909669
LOOKUPS['Ioue'] = Iuppiter
```

What is described above is the default role of lemmatizers in the LatinCy pipeline. Of course, it may be the case that you want to add additional lookups to your annotation pipeline overriding the default lookups. Here is an example of such a set up using a custom `Language.factory`. We will cover custom factories in more detail in a later notebook.

First, let's lemmatizer a sentence with some unexpected, non-Classical Latin words usind the default settings…

```
# cf. https://la.wikipedia.org/wiki/Star_Wars#Episodium_VI:_Iedi_reduces
doc = nlp("In planeta Tatooine, sunt Principissa Leia et Lucas Skywalker noviter Jedi factus

data = []

for token in doc:
    data.append([token.text, token.lemma_])
```

```
print("Before...", end="\n\n")
print(tabulate.tabulate(data, headers=['Text', 'Lemma']))
```

```
Before...

Text         Lemma
-----------  -----------
In           in
planeta      planeta
Tatooine     tatooine
,            ,
sunt         sum
Principissa  principissa
Leia         leius
et           et
Lucas        Lucas
Skywalker    Skywalker
nouiter      nouiter
Iedi         Iedus
factus       facio
```

Now we will lemmatize again but with a custom lookup table (i.e. `dict`) added to the pipeline…

```
from spacy import Language

custom_lemma_lookups = {
    "Jedi": ["Jedi"],
    "Tatooine": ["Tatooine"],
    "Leia": ["Leia", "Leiae", "Leiam"],
}

custom_lookups = {}
for key, values in custom_lemma_lookups.items():
    for value in values:
        if value not in LOOKUPS:
            custom_lookups[value] = key

@Language.component(name="custom_lookup_lemmatizer")
def make_lookup_lemmatizer_function(doc):
    for token in doc:
        token.lemma_ = custom_lookups.get(token.text, token.lemma_)
```

```
    return doc

try:
    nlp.add_pipe("custom_lookup_lemmatizer", name="custom_lookup_lemmatizer")
except:
    # If the pipeline component is already added, we can't add it again
    pass

# cf. https://la.wikipedia.org/wiki/Star_Wars#Episodium_VI:_Iedi_reduces
doc = nlp("In planeta Tatooine, sunt Principissa Leia et Lucas Skywalker noviter Jedi factus

data = []

for token in doc:
    data.append([token.text, token.lemma_])

print("After...", end="\n\n")
print(tabulate.tabulate(data, headers=['Text', 'Lemma']))
```

```
After...

Text         Lemma
-----------  -----------
In           in
planeta      planeta
Tatooine     Tatooine
,            ,
sunt         sum
Principissa  principissa
Leia         Leia
et           et
Lucas        Lucas
Skywalker    Skywalker
nouiter      nouiter
Iedi         Iedus
factus       facio
```

Note in particular how the default lemmatizers handled "Leia" without issue (being closer to an expected noun form ending in -*a*), but now better handles "Tatooine" and "Jedi" because of our custom lookup table.

References

NLTK Chapter 3, Section 3.6 "Normalizing text" link
SLP Chapter 2, Section 2.2 "Morphemes: Parts of Words" link
spaCy EditTreeLemmatizer link

7 POS Tagging

7.1 POS Tagging with LatinCy

Part-of-speech tagging is the task of mapping a token in a text to its part of speech, whether 'noun' or 'verb' or 'preposition' and so on. There are two components in the default LatinCy pipelines that provide such annotations, that is the `tagger` and the `morphologizer`. Ostensibly, the `tagger` provides language-specific, fine-grain POS tags and the `morphologizer` provides coarse-grain tags (as defined by the UD Universal POS tags); at present, the LatinCy models have a high degree of overlap between these two tagsets and there are effectively *no* fine-grain tags.

```
# Imports & setup

import spacy
from pprint import pprint
nlp = spacy.load('la_core_web_lg')
text = "Haec narrantur a poetis de Perseo. Perseus filius erat Iovis, maximi deorum. Avus eiu
doc = nlp(text)
print(doc)
```

```
Haec narrantur a poetis de Perseo . Perseus filius erat Iouis , maximi deorum . Auus eius Ac
```

Note here the two "tagging" components that are included in the pipeline, i.e. "tagger" and "morphologizer"…

```
print(nlp.pipe_names)
```

```
['tok2vec', 'senter', 'normer', 'tagger', 'morphologizer', 'trainable_lemmatizer', 'lookup_le
```

Once a text is annotated using the LatinCy pipeline, i.e. as part of the `Doc` creation process, tags can be found as annotations of the `Token` objects. The coarse-grain tags are stored in the `pos_` attribute; fine-grain tags are stored in the `tag_` attribute.

```
sample_token = doc[1]

print(f'Sample token: {sample_token.text}')
print(f'Sample POS: {sample_token.pos_}')
print(f'Sample TAG: {sample_token.tag_}')
```

```
Sample token: narrantur
Sample POS: VERB
Sample TAG: verb
```

Note the high degree of overlap between the coarse-grain and fine-grain tags in the LatinCy models in the chart below. That said, it is perhaps worth paying more attention to where the tagsets do not overlap. In the LatinCy training, conventions for classes of words to be labeled, say, "DET" (as in the *Haec* `pos_`) or "AUX" (as in *est*) are inferred from usage in the six different treebanks used. A sense of the inconsistency in the tagsets can be gleaned from the following page: (https://universaldependencies.org/la/); note also the important work of Gamba and Zeman (2023) in this area.

```
import tabulate

data = []

tokens = [item for item in doc]

for token in tokens:
    data.append([token.text, token.pos_, token.tag_])

print(tabulate.tabulate(data, headers=['Text', "POS", "TAG"]))
```

```
Text          POS    TAG
------------  -----  --------------------
Haec          DET    determiner
narrantur     VERB   verb
a             ADP    preposition/ablative
poetis        NOUN   noun
de            ADP    preposition/ablative
Perseo        PROPN  noun/proper
.             PUNCT  punc
Perseus       PROPN  noun/proper
filius        NOUN   noun
erat          AUX    verb
```

```
Iouis         PROPN  noun/proper
,             PUNCT  punc
maximi        ADJ    adjective
deorum        NOUN   noun
.             PUNCT  punc
Auus          PROPN  adjective
eius          PRON   pronoun
Acrisius      PROPN  noun/proper
appellabatur  VERB   verb
.             PUNCT  punc
```

As with the lemma annotations, the `pos_` and `tag_` attributes are only the human-readable of the lemma. Internally, spaCy uses a hash value to represent this, again noting the lack of trailing underscore…

```
print(f"Token: {tokens[1].text}")
print(f'Human-readable TAG: {tokens[1].tag_}')
print(f'spaCy TAG key: {tokens[1].tag}')
```

```
Token: narrantur
Human-readable TAG: verb
spaCy TAG key: 6360137228241296794
```

This tag 'key' can be looked up in spaCy's `NLP.vocab.strings` atrribute…

```
T = nlp.get_pipe('tagger')
tag_lookup = T.vocab.strings[6360137228241296794] # also would work on `nlp`, i.e. nlp.vocab

print(f'TAG key: {tokens[1].tag}')
print(f'Human-readable TAG: {tag_lookup}')
```

```
TAG key: 6360137228241296794
Human-readable TAG: verb
```

The same process applies to the POS 'keys'…

```
print(f'Token: {tokens[1].text}')
print(f'Human-readable POS: {tokens[1].pos_}')
print(f'spaCy POS key: {tokens[1].pos}')
```

```
Token: narrantur
Human-readable POS: VERB
spaCy POS key: 100
```

```
M = nlp.get_pipe("morphologizer")
pos_lookup = M.vocab.strings[100]
print(f'POS key: {tokens[1].pos}')
print(f'Human-readable POS: {pos_lookup}')
```

```
POS key: 100
Human-readable POS: VERB
```

We can use the `label_data` attribute from the `morphologizer` component to derive the complete (coarse-grain) tagset…

```
def split_pos(morph):
    if 'POS=' in morph:
        return morph.split('POS=')[1].split('|')[0]
    else:
        return None

tagset = sorted(list(set([split_pos(k) for k, v in M.label_data['morph'].items() if split_pos
print(tagset)
```

```
['ADJ', 'ADP', 'ADV', 'AUX', 'CCONJ', 'DET', 'INTJ', 'NOUN', 'NUM', 'PART', 'PRON', 'PROPN',
```

SpaCy has an `explain` methods that can show human-readable descriptions of these standard tags…

```
data = []

for tag in tagset:
    data.append([tag, spacy.explain(tag)])

print(tabulate.tabulate(data, headers=['TAG', 'Description']))
```

```
TAG    Description
-----  -------------------------
ADJ    adjective
ADP    adposition
```

```
ADV    adverb
AUX    auxiliary
CCONJ  coordinating conjunction
DET    determiner
INTJ   interjection
NOUN   noun
NUM    numeral
PART   particle
PRON   pronoun
PROPN  proper noun
PUNCT  punctuation
SCONJ  subordinating conjunction
VERB   verb
X      other
```

It may also be useful to know the "confidence" of the tagger in making its decision. We can derive this from the output tagger's `model.predict` method. This returns (at least, in part) a ranked list of per-scores, the maximum value of which determines the final annotation.

```python
# Helper function to get tagging scores

def get_tagging_scores(doc, n=3):
    # cf. https://stackoverflow.com/a/69228515
    scores = []
    tagger = nlp.get_pipe('tagger')
    labels = tagger.labels
    for token in doc:
        token_scores = tagger.model.predict([doc])[0][token.i]
        r = [*enumerate(token_scores)]
        r.sort(key=lambda x: x[1], reverse=True)
        scores.append([(labels[i], p) for i, p in r[:n]])
    return scores
```

```python
# Get the top 3 tags by score for each token in the Doc

tagging_probs = get_tagging_scores(doc)

for token in doc:
    print(f'Token: {token.text}', end='\n\n')
    data = []
    for label, prob in tagging_probs[token.i]:
        data.append([label, prob])
```

```
    print(tabulate.tabulate(data, headers=['Label', 'Score']))
    break
```

```
Token: Haec

Label          Score
----------  --------
determiner  12.5675
pronoun      5.07097
noun         2.13383
```

References

NLTK Chapter 5 "Categorizing and tagging words" link
SLP Chapter 17 "Sequence Labeling for Parts of Speech and Named Entities" link
spaCy Tagger and Morphologizer

8 Morphological Tagging

8.1 Morphological Tagging with LatinCy

Morphological tagging is the task of mapping a token in a text to various morphological tags as appropriate for the token's part of speech. A noun will have morphological tags for its gender, number, and case, while a verb—a finite verb, at least—will have tags for its person, number, tense, mood, and voice. The default LatinCy pipelines have a `morphologizer` component that assigns these tags.

```
# Imports & setup

import spacy
from pprint import pprint
nlp = spacy.load('la_core_web_lg')
text = "Haec narrantur a poetis de Perseo. Perseus filius erat Iovis, maximi deorum. Avus eiu
doc = nlp(text)
print(doc)
```

```
Haec narrantur a poetis de Perseo . Perseus filius erat Iouis , maximi deorum . Auus eius Ac
```

Morphological tags are annotations of the `Token` objects and are stored in the `morph` attribute.

```
sample_token = doc[1]

print(f'Sample token: {sample_token.text}')
print(f'Sample morph: {sample_token.morph}')
```

```
Sample token: narrantur
Sample morph: Aspect=Imp|Mood=Ind|Number=Plur|Person=3|Tense=Pres|VerbForm=Fin|Voice=Pass
```

Note that the morphological annotations are stored in a spaCy's `MorphAnalysis` object…

```
type(sample_token.morph)
```

```
spacy.tokens.morphanalysis.MorphAnalysis
```

LatinCy users may find the following `MorphAnalysis` methods of use.

To get all of the morphological tags for a token as a Python `dict`, you can use the `to_dict` method…

```
sample_morph_dict = sample_token.morph.to_dict()
pprint(sample_morph_dict)
```

```
{'Aspect': 'Imp',
 'Mood': 'Ind',
 'Number': 'Plur',
 'Person': '3',
 'Tense': 'Pres',
 'VerbForm': 'Fin',
 'Voice': 'Pass'}
```

You can also get the string representation of the morphological analysis with the `to_json` method:

```
sample_morph_dict = sample_token.morph.to_json()
pprint(sample_morph_dict)
```

```
'Aspect=Imp|Mood=Ind|Number=Plur|Person=3|Tense=Pres|VerbForm=Fin|Voice=Pass'
```

Conversely, a `MorphAnalysis` object can be created from a Python `dict` using the `set_morph` method on a spaCy `Token` object.

```
text = "Festina lente."
doc_no_annotations = nlp.make_doc(text)
print(f'{doc_no_annotations[0].text} -> {doc_no_annotations[0].morph if doc_no_annotations[0
```

```
Festina -> {No morph data}
```

```
festina_dict = {"Person": "2", "Number": "Singular", "Tense": "Present", "Mood": "Imperative"
doc_no_annotations[0].set_morph(festina_dict)
print(f'{doc_no_annotations[0].text} -> {doc_no_annotations[0].morph if doc_no_annotations[0
print(type(doc_no_annotations[0].morph))
```

```
Festina -> Mood=Imperative|Number=Singular|Person=2|Tense=Present|Voice=Active
<class 'spacy.tokens.morphanalysis.MorphAnalysis'>
```

8.1.1 The `remorpher` component

As noted above, the mophological tags are POS-specific. LatinCy uses a limited set of morphological keys based on the UD treebanks. Moreover, with respect to the tag values, one specific adjustment that has been made from the UD treebanks is that for verbs LatinCy can return the six "traditional" tense values, i.e. present, imperfect, future, perfect, pluperfect, and future perfect. These are mapped to the custom Token attributes by the 'remorpher' component and stored in the `._.remorph` attribute.

```
import tabulate

data = []

doc = nlp("Tum arcam ipsam in mare coniecit.")
tokens = [item for item in doc]

print('The morph for "coniecit" is...')
print(tokens[5].morph.to_json(), end='\n\n')

print('The remorph for "coniecit" is...')
print(tokens[5]._.remorph)
```

```
The morph for "coniecit" is...
Aspect=Perf|Mood=Ind|Number=Sing|Person=3|Tense=Past|VerbForm=Fin|Voice=Act

The remorph for "coniecit" is...
perfect
```

Note how in the sentence above the verb *coniecit* is tagged by the 'remorpher' with the 'Tense=Perf' (for perfect), whereas in the UD treebanks-derived morph attribute this is annotated with 'Tense=Past' in coordination with 'Aspect=Perf'.

References

NLTK Chapter 5 "Categorizing and tagging words" link
spaCy Morphologizer

9 Named Entity Recognition

9.1 Named Entity Recognition with LatinCy

Named entity recognition is the task of mapping a token (or span of tokens) in a text to a kind of categorical tag, e.g. mapping the token *Roma* to a tag indicating location. The default LatinCy pipelines have a `ner` component that assigns the following tags:

- `PERSON` for the names of people
- `LOC` for the names of locations
- `NORP` for the names of groups of people

```
# Imports & setup

import spacy
from pprint import pprint
nlp = spacy.load('la_core_web_lg')
# cf. Caesar, BC 3.1
text = "Dictatore habente comitia Caesare consules creantur Iulius Caesar et P. Servilius."
doc = nlp(text)
print(doc)
```

```
Dictatore habente comitia Caesare consules creantur Iulius Caesar et P . Seruilius .
```

```
import tabulate

data = []
tokens = [item for item in doc]

for token in tokens:
    data.append([token.text, token.ent_type_])

print(tabulate.tabulate(data, headers=['Text', "Entity Type"]))
```

```
Text       Entity Type
---------  -------------
Dictatore
habente
comitia
Caesare    PERSON
consules
creantur
Iulius     PERSON
Caesar     PERSON
et
P
.
Seruilius  PERSON
.
```

Named entities are annotated in the `Token` objects and are stored in the `ent_type_` attribute.

```
sample_token = doc[7]

print(f'Sample token: {sample_token.text}')
print(f'Sample entity type: {sample_token.ent_type_}')
```

```
Sample token: Caesar
Sample entity type: PERSON
```

Moreover, the ner component should be able to recover the entity spans from the text using IOB (i.e. inside/outside/beginning) annotations. The IOB annotations are stored in the `ent_iob_` attribute of a spaCy `Token` object.

In the example below, the PERSON named entity is "Julius Caesar". The word "et" is *not* part of the entity. Accordingly, "Julius" is tagged with `B-PERSON` (beginning of the entity), "Caesar" is tagged with `I-PERSON` (inside the entity, and in this case the last word in the entity), and "et" is tagged with `O` (outside the entity, and so closing off the preceding entity).

```
print(f'Sample token: {doc[6].text}')
print(f'Sample entity type: {doc[6].ent_type_}')
print(f'Sample entity IOB: {doc[6].ent_iob_}')
print(f"Sample entity IOB-type: {doc[6].ent_iob_}-{doc[6].ent_type_}", end='\n\n')

print(f'Sample token: {sample_token.text}')
```

```
print(f'Sample entity type: {sample_token.ent_type_}')
print(f'Sample entity IOB: {sample_token.ent_iob_}')
print(f"Sample entity IOB-type: {sample_token.ent_iob_}-{sample_token.ent_type_}", end='\n\n

print(f'Sample token: {doc[8].text}')
print(f'Sample entity type: {doc[8].ent_type_}')
print(f'Sample entity IOB: {doc[8].ent_iob_}')
```

```
Sample token: Iulius
Sample entity type: PERSON
Sample entity IOB: B
Sample entity IOB-type: B-PERSON

Sample token: Caesar
Sample entity type: PERSON
Sample entity IOB: I
Sample entity IOB-type: I-PERSON

Sample token: et
Sample entity type:
Sample entity IOB: O
```

LatinCy does not (yet) offer a spaCy EntityLinker, that is a model-based tagger for resolving named entities to specific referent in a knowledge base. The wikidata identifier for Julius Caesar is [https://www.wikidata.org/wiki/Q1048] and an EntityLinker (if it existed!) could annotate the entity span with this identifier. For now, it will suffice to note that spaCy `Token` objects have an attribute set aside for this purpose: `ent_kb_id_`.

```
sample_token = doc[7]
print(f'Sample token: {sample_token.text}')
print(f'Sample entity type: {sample_token.ent_type_}')
sample_token.ent_kb_id_ = "https://www.wikidata.org/wiki/Q1048"
print(f'Sample entity KB ID: {sample_token.ent_kb_id_}')
```

```
Sample token: Caesar
Sample entity type: PERSON
Sample entity KB ID: https://www.wikidata.org/wiki/Q1048
```

SpaCy has a visualizer available for named entities; this is covered in the displaCy notebook.

References

spaCy EntityRecognizer

10 Vectors

10.1 Accessing Vectors with LatinCy

```
# Imports & setup

import spacy
import numpy as np
from tabulate import tabulate
from pprint import pprint
nlp = spacy.load('la_core_web_lg')
text = "Haec narrantur a poetis de Perseo. Perseus filius erat Iovis, maximi deorum; avus eiu
doc = nlp(text)
print(doc)
```

```
Haec narrantur a poetis de Perseo . Perseus filius erat Iouis , maximi deorum ; auus eius Ac
```

Vectors are used in two different ways in LatinCy. First, a static embedding layer is used during training to initialize what will become a contextual embedding layer during training. Second, this contextual embedding layer—spaCy's tok2vec component—is used to produce the final token vectors that are used as input features for other components. LatinCy models use Floret vectors for the static embedding layer, both in the md and lg model sizes. The sm model size does not use static vectors at all, relying instead on hash embeddings of word-level features (normalized form, prefix, suffix, shape) to produce the contextual tok2vec output.

Again…

1. **Static floret vectors** (300 dimensions) — trained on the LatinCy texts corpus, these assign a fixed vector to each word form. Accessed via `token.vector`. The same word always receives the same vector, regardless of context.
2. **Contextual tok2vec vectors** (96 dimensions) — produced by a CNN that takes the static vectors as input and incorporates surrounding context. Accessed via `doc.tensor[token.i]`. The same word receives *different* vectors in different sentences.

New for v3.9: The `pretrain` initializes the model with information from a large amount of raw text so that the model can learn sentence structure, word cooccurrence, and other general language patterns.

Here is the vector for the first token 'Haec' in the text given above, output trunctated to the first 10 dimensions:

```
tokens = [token for token in doc]
for token in tokens:
    print(token.text)
    print(token.vector[:10], "etc.")
    break
```

```
Haec
[-0.7778918  -0.24657087 -2.321399   -4.946182   -0.47471917 -1.3058611
 -0.06238138 -1.4708582  -2.1147673  -2.00264   ] etc.
```

And here is a list of the first 10 tokens, each with the first 10 dimensions of their static floret vector…

```
vector_example_data = []
for token in tokens[:10]:
    vector_example_data.append((token.text, *token.vector[:10]))

print(tabulate(vector_example_data))
```

```
---------  ---------  ---------  ---------  ---------  ----------  ---------  ----------  ---
Haec       -0.777892  -0.246571  -2.3214    -4.94618   -0.474719   -1.30586   -
0.0623814  -1.47086     -2.11477   -2.00264
narrantur   1.44294    2.76409   -0.174809  -1.1091     1.32203     1.68776    1.97994     1
0.558864  -1.3482
a           3.0887     4.14845    4.40425   12.6695     3.4945      0.35588    0.8834      2
0.78953
poetis      2.54862    2.26177    2.09329   -0.416738   0.0676873   2.26123    1.91584     1
de         -2.47883    8.23127    2.65751   -0.477113  -0.854738   -3.52511   -
0.143562    4.56366      5.87061   -1.26645
Perseo      0.134013   0.504851  -2.4602     1.95082   -2.07618     0.364746  -
2.08651     0.00902766  -2.2649     0.958519
.           1.91543   10.3048    -8.6075    -3.56385    9.61896     3.71153   -
4.3554     -4.69565      5.341      2.3426
Perseus    -0.166723  -0.414044  -0.710938   2.38089   -1.03997     2.27024   -
0.865694    0.445143    -0.968434   1.27817
```

```
filius      0.104769  -1.6956    -1.11373    5.48569   -2.83982     0.680782  -
1.42932     1.20112      2.99276   -0.0684272
erat        5.0704     1.04352   -0.562308   3.80192   -1.54061    -2.32062   -
2.4345      1.86567      4.33458   -0.744219
---------  ---------  ---------  ---------  ---------  ----------  ---------  ----------  ---
```

We can get the vector length from the `shape` attribute…

```
print(doc[0].vector.shape)
```

```
(300,)
```

10.1.1 Static vectors are context-independent

Because `token.vector` returns static floret vectors, the same word form always produces the same vector — regardless of the sentence it appears in. Here we compare the vector for *erat* in two different sentences:

```
# Static floret vectors: same word → same vector
doc_a = nlp("Caesar magnus erat.")
doc_b = nlp("Tempus longum erat.")

erat_a = [t for t in doc_a if t.text == "erat"][0]
erat_b = [t for t in doc_b if t.text == "erat"][0]

print(f"'erat' in '{doc_a.text}':")
print(erat_a.vector[:10])
print(f"\n'erat' in '{doc_b.text}':")
print(erat_b.vector[:10])
print(f"\nVectors identical? {np.array_equal(erat_a.vector, erat_b.vector)}")
```

```
'erat' in 'Caesar magnus erat .':
[ 5.0704026   1.0435151  -0.56230766  3.801923   -1.5406077  -2.3206182
 -2.4345021   1.8656712   4.334582   -0.74421906]

'erat' in 'Tempus longum erat .':
[ 5.0704026   1.0435151  -0.56230766  3.801923   -1.5406077  -2.3206182
 -2.4345021   1.8656712   4.334582   -0.74421906]

Vectors identical? True
```

10.1.2 Contextual vectors from tok2vec

The tok2vec component on the other hand takes the static floret vectors as input and produces *contextual* representations that incorporate information from surrounding tokens. These are stored in `doc.tensor`, a matrix with one row per token, and have 96 dimensions in the `lg` model.

Unlike the static vectors, the contextual tok2vec output for the same word form *differs* depending on context, though note still the high similarity between the two vectors for *erat* in the different sentences (cosine similarity of ~0.97).

```
# Contextual tok2vec output: same word → different vector in different context
tensor_a = doc_a.tensor[erat_a.i]
tensor_b = doc_b.tensor[erat_b.i]

print(f"tok2vec output for 'erat' in '{doc_a.text}':")
print(tensor_a[:10])
print(f"\ntok2vec output for 'erat' in '{doc_b.text}':")
print(tensor_b[:10])
print(f"\nVectors identical? {np.array_equal(tensor_a, tensor_b)}")
cos_sim = np.dot(tensor_a, tensor_b) / (np.linalg.norm(tensor_a) * np.linalg.norm(tensor_b))
print(f"Cosine similarity: {cos_sim:.4f}")
```

```
tok2vec output for 'erat' in 'Caesar magnus erat .':
[-0.7828857   1.2530129   3.217065   -1.987933    1.0486498  -0.9898325
 -2.239715    0.518736   -0.5815717  -0.19825797]

tok2vec output for 'erat' in 'Tempus longum erat .':
[ 0.6583333   1.0896324   3.5292163   0.29009092  1.1463788  -1.0406121
 -1.8273878   0.71445364  1.443218   -1.7423413 ]

Vectors identical? False
Cosine similarity: 0.7458
```

10.1.3 Span and document vectors

The `span.vector` and `doc.vector` properties return the mean of the *static* `token.vector` values — not the contextual tok2vec output. These averaged static vectors provide a simple, deterministic representation for a span of text:

```
sents = [sent for sent in doc.sents]

for sent in sents:
    print(sent)
    print(sent.vector[:10])
    break
```

```
Haec narrantur a poetis de Perseo .
[ 0.83899707  3.9955199  -0.62983674  0.58676285  1.5853631   0.5071678
 -0.2669535   0.50204414  1.6783661  -0.22031358]
```

The `doc.similarity()` method computes cosine similarity between averaged static vectors. Because these are based on static floret embeddings, the similarity score reflects overlap in word forms and their trained associations, not context-dependent meaning:

```
text_1 = "Omnia vincit amor."
text_2 = "Omnia vincit amicitia."

doc_1 = nlp(text_1)
doc_2 = nlp(text_2)

print("\nSimilarity between the two sentences...")
similarity = doc_1.similarity(doc_2)
print(similarity)
```

```
Similarity between the two sentences...
0.9378001689910889
```

References

SLP Ch. 5, "Embeddings" link
SLP Ch. 10 "Masked Language Models" link

11 TRF Vectors

11.1 Accessing TRF Vectors with LatinCy

```
# Imports & setup

import spacy
import numpy as np
from tabulate import tabulate
from pprint import pprint
nlp = spacy.load('la_core_web_trf')
text = "Haec narrantur a poetis de Perseo. Perseus filius erat Iovis, maximi deorum; avus eiu
doc = nlp(text)
print(doc)
```

```
/Volumes/fiona/work/code/diy/latincy-v3/latincy-book/.venv/lib/python3.11/site-
packages/thinc/shims/pytorch.py:114: UserWarning: CUDA is not available or torch_xla is impor
  with torch.autocast(device_type="cuda", enabled=self._mixed_precision):

Haec narrantur a poetis de Perseo . Perseus filius erat Iouis , maximi deorum ; auus eius Ac
```

As noted in the previous notebook, the trf vectors work slightly differently than vectors in the other pipelines. This notebook covers the specifics of working with the trf contextual vectors.

Here is the vector for the first token 'Haec' in the text given above…

```
tokens = [token for token in doc]
for token in tokens:
    print(token.text)
    print(token.vector[:10], "etc.")
    break
```

```
Haec
[ 0.9139471  -0.5735928   0.28639206  0.24682833  1.6527059  -0.84686863
  1.3196969   0.68133825  1.2240002  -0.27457994] etc.
```

The LatinCy trf model uses MultilingualBERT as the basis of its 'transformer' component. The trf model has a custom Doc attribute that can give access to the per-token contextual vectors which is in turn mapped back to the `vector` attribute of the `Token` object via a user hook. Accordingly, you can just access the `vector` as you do with the other pipelines, but the output will be contextually informed.

```
# Here is an example using Acrisius, a word which appears twice in the given text...

print("Token.text values for two tokens...")
print(doc[17])
print(doc[20])

# Note that we have a Doc custom attribute `trf_token_vecs` which is a list of the vectors fo

print("\nSlices correspondiong to Acrisius from the trf_token_vecs Doc custom attribute...")
print(doc._.trf_token_vecs[17][:5])
print(doc._.trf_token_vecs[20][:5])

# But we can access these contextual vectors directly via the `vector` Token attribute.
print("\nToken attribute for the Acrisius slices...")
print(doc[17].vector[:5])
print(doc[20].vector[:5])

print("\nVectors are the same?")
print(np.mean(doc[17].vector) == np.mean(doc._.trf_token_vecs[17]))
```

```
Token.text values for two tokens...
Acrisius
Acrisius

Slices correspondiong to Acrisius from the trf_token_vecs Doc custom attribute...
[ 0.01897538 -1.0430044   3.0466883  -1.4863216   2.0580213 ]
[-0.06602303 -0.9354496   2.5874202  -0.9346686   2.0108151 ]

Token attribute for the Acrisius slices...
[ 0.01897538 -1.0430044   3.0466883  -1.4863216   2.0580213 ]
[-0.06602303 -0.9354496   2.5874202  -0.9346686   2.0108151 ]

Vectors are the same?
True
```

Reminder that the shape of the (MultilingualBERT-derived) vectors is 768…

```
print(doc[0].vector.shape)
```

```
(768,)
```

For the trf pipelines, I have (not yet!) implemented a `similarity` method. Vectors can of course still be compared using other methods, such as cosine similarity. Here is the cosine similarity between the vectors for two sentences using a custom script…

```
text_1 = "Omnia vincit amor."
text_2 = "Omnia vincit amicitia."

doc_1 = nlp(text_1)
doc_2 = nlp(text_2)

def cosine_similarity(vec1, vec2):
    # Ensure vectors are flattened numpy arrays
    vec1_flat = vec1.reshape(-1)
    vec2_flat = vec2.reshape(-1)

    # Compute cosine similarity
    dot_product = np.dot(vec1_flat, vec2_flat)
    norm1 = np.linalg.norm(vec1_flat)
    norm2 = np.linalg.norm(vec2_flat)

    return dot_product / (norm1 * norm2)

# Get the vectors and compute similarity
vec1 = doc_1._.trf_token_vecs
vec2 = doc_2._.trf_token_vecs

print("\nSimilarity between the two sentences...")
similarity = cosine_similarity(vec1, vec2)
print(similarity)
```

```
Similarity between the two sentences...
0.9443054
```

References

SLP Chapter 5, "Embeddings" link
SLP Ch. 10 "Masked Language Models" link

12 Noun Chunking

12.1 Noun Chunking with LatinCy

Noun chunking is the task of identifying a noun and the words directly related to it. More specifically, with respect to the LatinCy pipeline, it is using annotations of the POS-tagger (`tagger`) and the dependency parser (`parser`) to identify all children of a token tagged as NOUN. Unlike the NLP tasks in the preceding sections, this task cannot be associated directly with a pipeline component; rather it is a special `Span` case defined in the spaCy language model itself under `syntax iterators`.

```
# Imports & setup

import spacy
from pprint import pprint
nlp = spacy.load('la_core_web_lg')
text = "Haec narrantur a poetis de Perseo. Perseus filius erat Iovis, maximi deorum. Avus eiu
doc = nlp(text)
print(doc)
```

```
Haec narrantur a poetis de Perseo . Perseus filius erat Iouis , maximi deorum . Auus eius Ac
```

As defined in the `syntax_iterators`, noun chunks become attributes of the spaCy `Doc` object. Noun chunks will return any token span for which the dependency head has the `NOUN` tag. A one-word noun chunk is just the noun itself, so not of particular interest. So, really what we are looking for are longer spans of noun-related text.

```
# Print the noun chunks in the text
for noun_chunk in doc.noun_chunks:
    if len(noun_chunk) > 1:
        print(noun_chunk)
```

```
Perseus filius
maximi deorum
Auus eius
```

References

spaCy Noun chunks

13 Syntax iterators

13.1 Other kinds of "chunking" with LatinCy

As we saw in the previous notebooks, we can use the combination of annotations from the `tagger` and the `parser` to extract relevant spans of text, like noun chunks. Noun chunks are defined at the language level in spaCy—that is, in the "la" module itself—as a syntax iterator. (In fact, the spaCy documentation explicitly notes that the syntax iteration code is "at the moment, only used for noun chunks").

But the logic that is used to define a language-level syntax iterator can just as easily be used to write other such "chunk" extractors. In this notebook, we will look at two other kinds of "chunking" that we can do with LatinCy: *cum* clauses and prepositional phrases.

```
# Imports & setup

import spacy
from pprint import pprint
nlp = spacy.load('la_core_web_lg')

text = """Eo cum venisset, ad pedes Iasonis se proiecit et multis cum lacrimis eum obsecravi

doc = nlp(text)
print(doc)
```

```
Eo cum uenisset , ad pedes Iasonis se proiecit et multis cum lacrimis eum obsecrauit .
```

13.1.1 Extracting *cum* clauses with LatinCy

Writing syntax iterators can be more involved than other kinds of LatinCy text analysis. That said, it is still a good demonstration of how multiple annotations can work together to extract the kinds of syntactical structures that are of particular interest to Latinists whether in research or in teaching. We will break down the process as follows:

1. First, let's write a syntax iterator that will find (optimistically!) all subordinate clauses. (Note that at this stage in LatinCy development, parser performance, even on the best performing models, is far from perfect. Since determining where subordinate clauses begin and end corresponds directly to parser performance, by extension clause detection performance may be similarly imperfect.)
2. Second, we will write a function that will only return those subordinate clauses that are introduced by the subordinating conjunction *cum*. To test for this, we will look for a token anywhere in the clause that has both the lemma "cum" and the POS tag "SCONJ".

Note some details about the assumptions of this syntax iteration code…

- It assumes that a clause only contains one 'mark' annotation from the dependency parser, i.e. one word like a relative pronoun or a subordinating conjunction that marks the beginning of the clause.
- It removes both initial and final punctuation from the clause.

```
# Function for finding clauses

from spacy.symbols import VERB, PUNCT
from spacy.tokens import Doc

def find_clauses(doclike):
    """Extract clauses from Latin text by finding finite verbs within sentences."""

    def get_clause_tokens(verb_token):
        """Get all tokens in the clause by walking the dependency tree."""
        clause_tokens = set([verb_token])

        def collect_dependents(token):
            for child in token.children:
                # Stop at new clause boundaries
                if ((child.pos == VERB and child.morph.get("VerbForm") == ["Fin"]) or
                    child.dep_ == "cc"):
                    continue
                clause_tokens.add(child)
                collect_dependents(child)

        collect_dependents(verb_token)
        return clause_tokens

    doc = doclike.doc

    clause_label = doc.vocab.strings.add("CLAUSE")
```

```
    # Iterate through sentences and extract clauses
    for sent in doc.sents:
        finite_verbs = [t for t in sent
                        if t.pos == VERB and t.morph.get("VerbForm") == ["Fin"]]

        for verb in finite_verbs:
            clause_tokens = get_clause_tokens(verb)
            if clause_tokens:
                # Get initial bounds, i.e. find the left and right side of the clause
                left = min(t.i for t in clause_tokens)
                right = max(t.i for t in clause_tokens)

                # Check number of markers in the clause
                span = doc[left:right + 1]
                markers = [t for t in span if t.dep_ == "mark"]
                if len(markers) != 1:  # Only yield if exactly one marker
                    continue

                # Adjust bounds to skip punctuation
                while left < right and doc[left].pos == PUNCT:
                    left += 1
                while right > left and doc[right].pos == PUNCT:
                    right -= 1

                yield left, right + 1, clause_label
```

```
# Process text
doc = nlp(text)

# Find and display clauses
print(f'Here are the subordinate clauses in the text "{text}":')

doc.spans["clauses"] = [doc[start:end] for start, end, _ in find_clauses(doc)]
for span in doc.spans["clauses"]:
    print(f"- {span.text}")
    print(f"  Clause verb: {[t for t in span if t.pos == VERB][0].text}")
    print(f'  Introductory word: {[t for t in span if t.dep_ == "mark"][0].lemma_}')
```

```
Here are the subordinate clauses in the text "Eo cum venisset, ad pedes Iasonis se proiecit e
- Eo cum uenisset
  Clause verb: uenisset
```

```
  Introductory word: cum
```

Now, we write a function that extracts all of the subordinate clauses from a `Doc` object and limits its output to only those that meet our definition of a *cum* clause.

```
# Function for finding cum clauses

def find_cum_clauses(doclike):
    """Extract 'cum' clauses from Latin text by finding clauses containing lemma 'cum' as SC

    doc = doclike.doc
    cum_label = doc.vocab.strings.add("CUM")

    # Get all clauses first
    all_clauses = list(find_clauses(doc))

    # Filter for clauses containing 'cum' as SCONJ anywhere in the clause
    for start, end, _ in all_clauses:
        span = doc[start:end]
        # Look for 'cum' anywhere in the clause
        has_cum = any(token.lemma_ == "cum" and token.pos_ == "SCONJ" for token in span)

        if has_cum:
            yield start, end, cum_label
```

```
# Process text
text = """Eo cum venisset, ad pedes Iasonis se proiecit et multis cum lacrimis eum obsecravi

doc = nlp(text)

# Find and display clauses
print(f'Here are the cum clauses in the text "{text}":')

doc.spans["clauses"] = [doc[start:end] for start, end, _ in find_cum_clauses(doc)]
for span in doc.spans["clauses"]:
    print(f"- {span.text}")
    print(f"  Clause verb: {[t for t in span if t.pos == VERB][0].text}")
    print(f'  Introductory word: {[t for t in span if t.dep_ == "mark"][0].lemma_}')
```

```
Here are the cum clauses in the text "Eo cum venisset, ad pedes Iasonis se proiecit et multis
- Eo cum uenisset
  Clause verb: uenisset
```

```
Introductory word: cum
```

The advantage of using the syntax iterator—assuming that the dependency parser works well enough to correctly mark off clauses—is that it extracts more directly the syntactical structures that we are interested in. We may have tried a text analytical approach for this problem that started with a search for *cum*. Of course, *cum* can be both a marker of subordination (i.e. when it has the POS tag `SCONJ`) and a preposition (i.e. `ADP`). In the sample sentence, the iterator never even considers the prepositional uses of *cum* (in *multis cum lacrimis*).

```python
# List annotations for each token

doc = nlp(text)

data = []

for token in doc:
    data.append([
        token.text,
        token.lemma_,
        token.pos_,
        token.dep_,
    ])

from tabulate import tabulate

print(tabulate(data, headers=['Token', 'Lemma', 'POS', "Dep"]))
```

```
Token       Lemma    POS    Dep
----------  -------  -----  -----------
Eo          eo       ADV    advmod:lmod
cum         cum      SCONJ  mark
uenisset    uenio    VERB   advcl
,           ,        PUNCT  punct
ad          ad       ADP    case
pedes       pes      NOUN   obl
Iasonis     Iason    PROPN  nmod
se          sui      PRON   obj
proiecit    proicio  VERB   ROOT
et          et       CCONJ  cc
multis      multus   DET    amod
cum         cum      ADP    case
lacrimis    lacrima  NOUN   obl
```

```
eum         is       PRON   obj
obsecrauit  obsecro  VERB   conj
.           .        PUNCT  punct
```

13.1.2 Extracting prepositional phrases with LatinCy

As we have already seen with noun chunking, there are syntactic structures that we may want to extract that are not at the clause level. Take, for example, prepositional phrases. In a sense, these phrases are the preposition equivalent of noun chunks. Our job is to identify the preposition and all of the words that "depend" on it (from a dependency parsing perspective). We can start by finding words tagged as prepositions and then looking for all of the child nodes of these words.

```python
# Syntax iterator code for prepositional phrases

from typing import Iterator, Tuple, Union, Set
from spacy.symbols import ADP, PUNCT
from spacy.tokens import Doc, Span

def prepositional_phrases(doclike: Union[Doc, Span]) -> Iterator[Tuple[int, int, int]]:
    """Find prepositional phrases by identifying prepositions and their objects."""

    def get_phrase_tokens(prep_token) -> Set:
        """Get all tokens in the prepositional phrase by walking the dependency tree."""
        phrase_tokens = set([prep_token])

        # Get the noun that this preposition modifies (its head)
        if prep_token.dep_ == "case" and prep_token.head.pos_ in ["NOUN", "PROPN", "PRON"]:
            noun = prep_token.head
            phrase_tokens.add(noun)

            # Get all dependents of the noun
            for child in noun.children:
                if child != prep_token:  # avoid adding preposition twice
                    phrase_tokens.add(child)

        return phrase_tokens

    doc = doclike.doc

    if not doc.has_annotation("DEP"):
        raise ValueError(Errors.E029)
```

```
    if not len(doc):
        return

    pp_label = doc.vocab.strings.add("PP")
    prev_right = -1

    for token in doclike:
        if token.pos == ADP and token.dep_ == "case":
            phrase_tokens = get_phrase_tokens(token)
            if phrase_tokens:
                left = min(t.i for t in phrase_tokens)
                right = max(t.i for t in phrase_tokens)

                # Skip if overlaps with previous phrase
                if left <= prev_right:
                    continue

                # Adjust bounds to skip punctuation
                while left < right and doc[left].pos == PUNCT:
                    left += 1
                while right > left and doc[right].pos == PUNCT:
                    right -= 1

                yield left, right + 1, pp_label
                prev_right = right
```

```
# Helper function for displaying prepositional phrases

def display_phrases_markdown(doc: Doc) -> str:
    """Display text with prepositional phrases marked in bold."""
    # Get prepositional phrases
    spans = [doc[start:end] for start, end, _ in prepositional_phrases(doc)]

    # Build output from doc tokens, inserting bold markers around PP spans
    pp_starts = {span.start for span in spans}
    pp_ends = {span.end for span in spans}

    parts = []
    for token in doc:
        if token.i in pp_starts:
            parts.append("**")
```

```
        parts.append(token.text_with_ws)
        if token.i + 1 in pp_ends:
            # Insert closing marker before the trailing whitespace
            text = parts[-1]
            stripped = text.rstrip()
            ws = text[len(stripped):]
            parts[-1] = stripped + "**" + ws

    return "".join(parts)

# Test with passage from RFF

test_texts = [
    "Haec narrantur a poetis de Perseo.",
    "Acrisius volebat Perseum nepotem suum necare; nam propter oraculum puerum timebat.",
    "Comprehendit igitur Perseum adhuc infantem, et cum matre in arca lignea inclusit.",
    "Tum arcam ipsam in mare coniecit.",
    "Perseus autem in sinu matris dormiebat."
]

# Display results
print("Prepositional phrases marked with asterisks...")
for text in test_texts:
    doc = nlp(text)
    marked = display_phrases_markdown(doc)
    print(f"- {marked}")
```

```
Prepositional phrases marked with asterisks...
- Haec narrantur **a poetis de Perseo** .
- Acrisius uolebat Perseum nepotem suum necare ; nam **propter oraculum** puerum timebat .
- Comprehendit igitur Perseum adhuc infantem , et **cum matre** **in arca lignea** inclusit
- Tum arcam ipsam **in mare** coniecit .
- Perseus autem **in sinu matris** dormiebat .
```

Cum clauses and prepositional phrases are not the only type of syntactical structure that Latinists are interested. I invite readers to think about what the formal definitions with respect to LatinCy annotations would be for other kinds of structures, e.g. ablative absolutes, indirect statement, purpose clauses, etc., and see if they can be productively extracted using this kind of syntax iteration.

Part IV

spaCy Tasks

14 Sequence Matching

14.1 Sequence matching with LatinCy

We can use LatinCy annotations as the basis for matching spans of tokens using spaCy's `Matcher`. This includes basic `Token` attributes like `ORTH`, `TEXT`, `NORM`, and `LOWER` as well as those annotated by the LatinCy pipeline like `LEMMA`, `POS`, `TAG`, `MORPH`, `DEP`, and `ENT_TYPE`. There are also more general attributes like `IS_ALPHA`, `IS_ASCII`, `IS_DIGIT`, `IS_LOWER`, `IS_UPPER`, `IS_TITLE`, `IS_PUNCT`, `IS_SPACE`, and `LIKE_NUM`, among still others. The full list can be found here. Moreover, there are many operators, quantifiers, and other operations that can be using to create ever-increasingly complex patterns. The combinatorial patterns for using the `Matcher` are frankly enormous and so this should be considered only a most basic introduction for what is possible.

Here we run through a quick example based on finding variations of *res* and then by pattern-matching extensions of *res publica* using the *Praefatio* to Livy's *Ab urbe condita*.

```
# Imports & setup

import spacy
from pprint import pprint
from tabulate import tabulate

nlp = spacy.load('la_core_web_lg')

with open('livy_praefatio.txt') as f:
    text = f.read()

doc = nlp(text)
print(doc[:100])
```

```
Facturusne operae pretium sim si a primordio urbis res populi Romani perscripserim nec satis
```

The `Matcher` is initialized with the `Vocab` object from our loaded pipeline.

```
from spacy.matcher import Matcher

matcher = Matcher(nlp.vocab)
print(matcher)
```

```
<spacy.matcher.matcher.Matcher object at 0x122d32f80>
```

We use the `Matcher` by adding patterns, quite sensibly using the `add` method. With the `add` method, we assign a `match_id` "name" for the pattern and the patterns themselves. The patters are lists of lists of dictionaries; those dictionaries are arranged sequentially by the tokens sequences we want to match, where the dictionary keys are the attributes and the dictionary values are the our specific terms to be matched for that attribute. So, in the example below, we are looking for any span of tokens in the provided `Doc` where the `text` attribute matches—and matches exactly—the string "res". The `Matcher` returns the `match_id` as well as the start and end indices for each matched span.

```
pattern = [{'TEXT': 'res'}]
matcher.add('res_tokens', [pattern])

matches = matcher(doc)

matches_data = []

for match_id, start, end in matches:
    string_id = nlp.vocab.strings[match_id]
    span = doc[start:end]
    matches_data.append((string_id, start, end, span.text))

print(tabulate(matches_data, headers=['Match ID', 'Start', 'End', 'Matched text']))
```

```
Match ID      Start    End  Matched text
----------  -------  -----  --------------
res_tokens        8      9  res
res_tokens      422    423  res
```

```
# Helper functions

def pattern2matches(pattern_name, pattern):
    matcher = Matcher(nlp.vocab)
    matcher.add(pattern_name, [pattern])
```

```
    matches = matcher(doc)
    matches_data = []
    for match_id, start, end in matches:
        string_id = nlp.vocab.strings[match_id]
        span = doc[start:end]
        matches_data.append((string_id, start, end, span.text))

    return matches_data

def tabulate_matches(pattern_name, pattern):
    matches_data = pattern2matches(pattern_name, pattern)
    print(tabulate(matches_data, headers=['Match ID', 'Start', 'End', 'Matched text']))
```

Note that the matches above on `TEXT` are case-sensitive. We can widen our search for *res* by using the `LOWER` attribute…

```
pattern = [{'LOWER': 'res'}]
tabulate_matches('res_uncased', pattern)
```

```
Match ID       Start    End  Matched text
-----------  -------  -----  --------------
res_uncased        8      9  res
res_uncased       93     94  Res
res_uncased      422    423  res
```

Extending this logic even further, we can widen the search again by matching not only the token "Res"/"res" but all tokens for which the LatinCy lemmatizers have assigned the lemma "res". This is done by using the `LEMMA` attribute.

```
pattern = [{'LEMMA': 'res'}]
tabulate_matches('res_lemma', pattern)
```

```
Match ID      Start    End  Matched text
----------  -------  -----  --------------
res_lemma         8      9  res
res_lemma        30     31  rem
res_lemma        39     40  rebus
res_lemma        57     58  rerum
res_lemma        93     94  Res
res_lemma       213    214  rerum
```

```
res_lemma       378    379  rerum
res_lemma       397    398  rei
res_lemma       422    423  res
res_lemma       461    462  rerum
res_lemma       504    505  rei
```

So far, all of our patterns have included only a single token. We can extend our search to include multiple sequential tokens by adding more dictionaries to the list of dictionaries. In the example below, we are looking for any span where the first token has the lemma "res" and is followed by any token with the `POS` of "NOUN".

```
pattern = [{'LEMMA': 'res'}, {'POS': 'NOUN'}]
tabulate_matches('res_lemma_noun', pattern)
```

```
Match ID          Start    End  Matched text
--------------  -------  -----  --------------
res_lemma_noun        8     10  res populi
```

By contrast, we can return a span where the first token has the lemma "res" and is *not* followed by any token with the `POS` of "NOUN".

```
pattern = [{'LEMMA': 'res'}, {'TAG': 'NOUN', "OP": "!"}]
tabulate_matches('res_lemma_noun_not', pattern)
```

```
Match ID              Start    End  Matched text
------------------  -------  -----  --------------
res_lemma_noun_not        8     10  res populi
res_lemma_noun_not       30     32  rem uideam
res_lemma_noun_not       39     41  rebus certius
res_lemma_noun_not       57     59  rerum gestarum
res_lemma_noun_not       93     95  Res est
res_lemma_noun_not      213    215  rerum gestarum
res_lemma_noun_not      378    380  rerum salubre
res_lemma_noun_not      397    399  rei publicae
res_lemma_noun_not      422    424  res publica
res_lemma_noun_not      461    463  rerum minus
res_lemma_noun_not      504    506  rei absint
```

The `Matcher` allows for "fuzzy" matching based on Levenshtein distance (details in documentation, linked below)…

```
pattern = [{'LEMMA': 'res'}, {"TEXT": {"FUZZY": "public"}}]
tabulate_matches('res_public_fuzzy', pattern)
```

```
Match ID            Start    End  Matched text
----------------  -------  -----  --------------
res_public_fuzzy      397    399  rei publicae
res_public_fuzzy      422    424  res publica
```

Of course, you could search for *res publica* more directly with a two-lemma pattern…

```
pattern = [{'LEMMA': 'res'}, {'LEMMA': 'publicus'}]
tabulate_matches('res_publica_lemmas', pattern)
```

```
Match ID              Start    End  Matched text
------------------  -------  -----  --------------
res_publica_lemmas      397    399  rei publicae
res_publica_lemmas      422    424  res publica
```

The patterns are not limited to text search, that is they can be based entirely on annotation patterns. Here is a list of all NOUN-ADJ sequences in Livy's "Praefatio"…

```
pattern = [{"POS": "NOUN"}, {"POS": "ADJ"}]
tabulate_matches('noun_adjs', pattern)
```

```
Match ID      Start    End  Matched text
----------  -------  -----  -------------------
noun_adjs         9     11  populi Romani
noun_adjs        39     41  rebus certius
noun_adjs        46     48  arte rudem
noun_adjs       127    129  origines proxima
noun_adjs       130    132  originibus minus
noun_adjs       206    208  urbem poeticis
noun_adjs       236    238  urbium augustiora
noun_adjs       259    261  populo Romano
noun_adjs       275    277  gentes humanae
noun_adjs       378    380  rerum salubre
noun_adjs       397    399  rei publicae
noun_adjs       405    407  inceptu foedum
noun_adjs       422    424  res publica
noun_adjs       430    432  exemplis ditior
noun_adjs       536    538  successus prosperos
```

As well as a list of all alliterative patterns, though with some creative regexing…

```
matcher = Matcher(nlp.vocab)

for letter in "abcdefghijklmnopqrstuvwxyz":
    pattern = [{"LOWER": {"REGEX": rf'\b{letter}.+?\b'}, "OP": "{2,}"}]
    matcher.add('alliterative_pairs', [pattern])

matches = matcher(doc)

matches_data = []

for match_id, start, end in matches:
    string_id = nlp.vocab.strings[match_id]
    span = doc[start:end]
    matches_data.append((string_id, start, end, span.text))

print(tabulate(matches_data, headers=['Match ID', 'Start', 'End', 'Matched text']))
```

Match ID	Start	End	Matched text
alliterative_pairs	3	5	sim si
alliterative_pairs	13	15	satis scio
alliterative_pairs	17	19	si sciam
alliterative_pairs	23	25	quippe qui
alliterative_pairs	35	37	semper scriptores
alliterative_pairs	41	43	aliquid allaturos
alliterative_pairs	62	64	populi pro
alliterative_pairs	102	104	supra septingentesimum
alliterative_pairs	142	144	pridem praeualentis
alliterative_pairs	142	145	pridem praeualentis populi
alliterative_pairs	143	145	praeualentis populi
alliterative_pairs	155	157	praemium petam
alliterative_pairs	204	206	conditam condendamue
alliterative_pairs	278	280	aequo animo
alliterative_pairs	290	292	animaduersa aut
alliterative_pairs	292	294	existimata erunt
alliterative_pairs	346	348	magis magis
alliterative_pairs	364	366	nostra nec
alliterative_pairs	367	369	pati possumus
alliterative_pairs	367	370	pati possumus peruentum
alliterative_pairs	368	370	possumus peruentum

```
alliterative_pairs      387    389  in inlustri
alliterative_pairs      394    396  tibi tuae
alliterative_pairs      480    482  pereundi perdendi
alliterative_pairs      515    517  deorum dearum
```

References

spaCy Matcher

15 Visualizing Annotations

15.1 Visualizing Annotations with DisplaCy

Throughout this book, we have annotated many texts and often reported these annotations in simple lists and tables. But it can be useful for for some tasks to *see* the annotations and how they relate to our texts. SpaCy includes some basic visualization tools through a package called `displacy` that can help us accomplish this goal.

There are two primary types of annotations that `displaCy` can help us visualize: dependency parses and named entities. This notebook demonstrates both types of annotations.

```
# Imports & setup

import spacy
from tabulate import tabulate

from pprint import pprint
nlp = spacy.load('la_core_web_lg')
text = "Haec narrantur a poetis de Perseo. Perseus filius erat Iovis, maximi deorum. Avus eiu
doc = nlp(text)
print(doc)
```

```
Haec narrantur a poetis de Perseo . Perseus filius erat Iouis , maximi deorum . Auus eius Ac
```

15.1.1 Visualizing dependency parses

```
# import

from spacy import displacy
```

Let's take the first sentence from our sample text and list the relevant dependency annotations…

```
sents = list(doc.sents)

data = []

for token in sents[0]:
    data.append([token.i, token.text, token.dep_, token.head.text])

print(tabulate(data, headers=['','Token', 'Dep', 'Head']))
```

```
    Token      Dep         Head
--  ---------  ----------  ---------
 0  Haec       nsubj:pass  narrantur
 1  narrantur  ROOT        narrantur
 2  a          case        poetis
 3  poetis     obl         narrantur
 4  de         case        Perseo
 5  Perseo     obl         narrantur
 6  .          conj        narrantur
```

The table above reflects all of the relevant dependencies, but again in a list format, it can be hard to trace these relationships. Here is where a visualization can be useful…

```
# visualize dependency parse

displacy.render(sents[0], style='dep', jupyter=True)
```

```
<IPython.core.display.HTML object>
```

15.1.2 Visualizing named entities

We can also use displaCy to visualize named entity annotations. As we read in the previous chapter on the NER component, the LatinCy models currently support named entity annotations for the following entity types: `PERSON`, `LOC`, and `NORP` (i.e. groups of people). As with the dependency parses, let's first present entity annotations in a table format…

```
# Extract entities

text = """Iason et Medea e Thessalia expulsi ad urbem Corinthum venerunt, cuius urbis Creon
text =  text.replace("v","u").replace("V","U")
```

```python
doc = nlp(text)

data = []
tokens = [item for item in doc]

for token in tokens:
    data.append([token.text, token.ent_type_])

print(tabulate(data, headers=['Text', "Entity Type"]))
```

```
Text       Entity Type
---------  -------------
Iason      PERSON
et
Medea      PERSON
e
Thessalia  LOC
expulsi
ad
urbem
Corinthum  LOC
uenerunt
,
cuius
urbis
Creon      PERSON
quidam
regnum
tum
obtinebat
.
```

The disadvantages of this type of presentation are not hard to notice. It lacks compactness and perhaps more importantly there is a lack of clear visual distinction between the different entity types. By contrast, displaCy can show us the entities in clear running text with each entity type color-coded for maximum visual impact.

```python
# Visualizing named entities

displacy.render(doc, style="ent", jupyter=True)
```

```
<IPython.core.display.HTML object>
```

References

spaCy Visualizers
MS Chapter 1 “Getting started with SpaCy”, section “Visualization with displaCy”

A Open LatinCy Projects

Listed below are some open projects that I would like to see implemented in the LatinCy pipelines. If you are interested in contributing to the development of these projects or have recommendations for additional projects that could be added, please contact me through the LatinCy GitHub or HuggingFace repositories.

A.1 Model-based enclitic splitting

As discussed in the Word Tokenization chapter, enclitic splitting is currently limited to *que* (and variants) and is a rules-based process that is part of the LatinCy custom tokenizer. It would be preferable to make enclitic splitting a separate pipeline component and moreover one that is model-based. This component could be placed immediately after the tokenizer and use the `Retokenizer.split` method to reconstruct token sequences where valid enclitics are identified. This would have the added advantage of allowing enclitic splitting to be "turned off", so to speak, by removing the component from the pipeline.

B Sample Texts

B.1 Livy, Ab Urbe Condita "Praefatio"

Facturusne operae pretium sim si a primordio urbis res populi Romani perscripserim nec satis scio nec, si sciam, dicere ausim, quippe qui cum veterem tum volgatam esse rem videam, dum novi semper scriptores aut in rebus certius aliquid allaturos se aut scribendi arte rudem vetustatem superaturos credunt. Utcumque erit, iuvabit tamen rerum gestarum memoriae principis terrarum populi pro virili parte et ipsum consuluisse; et si in tanta scriptorum turba mea fama in obscuro sit, nobilitate ac magnitudine eorum me qui nomini officient meo consoler. Res est praeterea et immensi operis, ut quae supra septingentesimum annum repetatur et quae ab exiguis profecta initiis eo creverit ut iam magnitudine laboret sua; et legentium plerisque haud dubito quin primae origines proximaque originibus minus praebitura voluptatis sint, festinantibus ad haec nova quibus iam pridem praevalentis populi vires se ipsae conficiunt: ego contra hoc quoque laboris praemium petam, ut me a conspectu malorum quae nostra tot per annos vidit aetas, tantisper certe dum prisca [tota] illa mente repeto, avertam, omnis expers curae quae scribentis animum, etsi non flectere a uero, sollicitum tamen efficere posset.

Quae ante conditam condendamve urbem poeticis magis decora fabulis quam incorruptis rerum gestarum monumentis traduntur, ea nec adfirmare nec refellere in animo est. Datur haec venia antiquitati ut miscendo humana divinis primordia urbium augustiora faciat; et si cui populo licere oportet consecrare origines suas et ad deos referre auctores, ea belli gloria est populo Romano ut cum suum conditorisque sui parentem Martem potissimum ferat, tam et hoc gentes humanae patiantur aequo animo quam imperium patiuntur. Sed haec et his similia utcumque animaduersa aut existimata erunt haud in magno equidem ponam discrimine: ad illa mihi pro se quisque acriter intendat animum, quae vita, qui mores fuerint, per quos viros quibusque artibus domi militiaeque et partum et auctum imperium sit; labente deinde paulatim disciplina velut desidentes primo mores sequatur animo, deinde ut magis magisque lapsi sint, tum ire coeperint praecipites, donec ad haec tempora quibus nec vitia nostra nec remedia pati possumus perventum est.

Hoc illud est praecipue in cognitione rerum salubre ac frugiferum, omnis te exempli documenta in inlustri posita monumento intueri; inde tibi tuaeque rei publicae quod imitere capias, inde foedum inceptu foedum exitu quod vites. Ceterum aut me amor negotii suscepti fallit, aut nulla unquam res publica nec maior nec sanctior nec bonis exemplis ditior fuit, nec in quam [civitatem] tam serae avaritia luxuriaque immigraverint, nec ubi tantus ac tam diu paupertati ac parsimoniae honos fuerit. Adeo quanto rerum minus, tanto minus cupiditatis erat: nuper

divitiae avaritiam et abundantes voluptates desiderium per luxum atque libidinem pereundi perdendique omnia invexere.

Sed querellae, ne tum quidem gratae futurae cum forsitan necessariae erunt, ab initio certe tantae ordiendae rei absint: cum bonis potius ominibus votisque et precationibus deorum dearumque, si, ut poetis, nobis quoque mos esset, libentius inciperemus, ut orsis tantum operis successus prosperos darent.

References

Bird, Steven, Ewan Klein, and Edward Loper. 2015. *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. 2nd Edition. https://www.nltk.org/book/.

Burns, Patrick J. 2018. "Backoff Lemmatization as a Philological Method." In *Digital Humanities 2018, DH 2018, Book of Abstracts, El Colegio de México, UNAM, and RedHD, Mexico City, Mexico, June 26-29, 2018*.

———. 2020. "Ensemble Lemmatization with the Classical Language Toolkit." *Studi e Saggi Linguistici* 58 (1): 157–76. https://doi.org/10.4454/ssl.v58i1.273.

Clayman, Dee. 1981. "Sentence Length in Greek Hexameter Poetry." *Quantitative Linguistics* 11: 107–36. https://papers.ssrn.com/abstract=1627358.

Gamba, Federica, and Daniel Zeman. 2023. "Universalising Latin Universal Dependencies: A Harmonisation of Latin Treebanks in UD." In *Proceedings of the Sixth Workshop on Universal Dependencies (UDW, GURT/SyntaxFest 2023)*, edited by Loïc Grobol and Francis Tyers, 7–16. Washington, D.C.: Association for Computational Linguistics. https://aclanthology.org/2023.udw-1.2.

Janson, Tore. 1964. "The Problems of Measuring Sentence-Length in Classical Texts." *Studia Linguistica* 18 (1): 26–36. https://doi.org/10.1111/j.1467-9582.1964.tb00443.x.

Jurafsky, Daniel, and James H. Martin. 2026. "Speech and Language Processing (3rd Edition, Draft)." https://web.stanford.edu/~jurafsky/slp3/.

Mesquita, Daniel, and Duygu Altinok. 2025. *Mastering spaCy*. 2nd ed. Birmingham, U.K.: Packt Publishing Ltd.

Wake, William C. 1957. "Sentence-Length Distributions of Greek Authors." *Journal of the Royal Statistical Society. Series A (General)* 120 (3): 331–46. https://www.jstor.org/stable/2343104.